

Advanced JUnit Testing Exercises

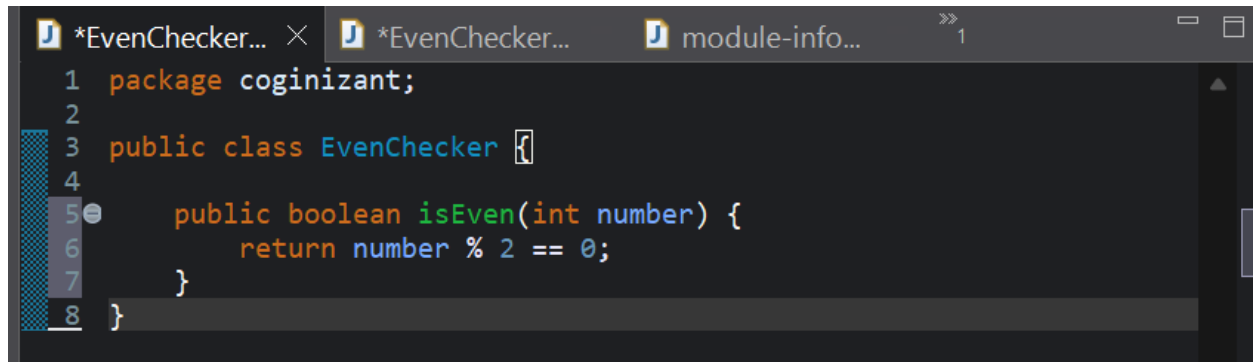
Exercise 1: Parameterized Tests

Scenario:

You want to test a method that checks if a number is even. Instead of writing multiple test cases, you will use parameterized tests to run the same test with different inputs.

Steps:

1. Create a new Java class `EvenChecker` with a method `isEven(int number)`.
2. Write a parameterized test class `EvenCheckerTest` that tests the `isEven` method with different inputs.
3. Use JUnit's `@ParameterizedTest` and `@ValueSource` annotations.



```
1 package coginizant;
2
3 public class EvenChecker {
4
5     public boolean isEven(int number) {
6         return number % 2 == 0;
7     }
8 }
```

EvenCheckerTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
public class EvenCheckerTest {
    EvenChecker checker = new EvenChecker();
    @ParameterizedTest
    @ValueSource(ints = {2, 4, 6, 8, 10})
    void testEvenNumbers(int number) {
        assertEquals(true, checker.isEven(number));
    }
    @ParameterizedTest
    @ValueSource(ints = {1, 3, 5, 7, 9})
    void testOddNumbers(int number) {
        assertEquals(false, checker.isEven(number));
    }
}
```

Exercise 2: Test Suites and Categories

Scenario:

You want to group related tests into a test suite and categorize them.

Steps:

1. Create a new test suite class `AllTests`.
2. Add multiple test classes to the suite.

3. Use JUnit's `@Suite` and `@SelectClasses` annotations.

Calculator.java

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
    public int multiply(int a, int b) {  
        return a * b;  
    }  
}
```

AddTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;  
import org.junit.jupiter.api.Test;  
public class AddTest {  
    Calculator c = new Calculator();  
    @Test  
    void testAdd() {  
        assertEquals(8, c.add(5,3));  
    }  
}
```

MultiplyTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;  
import org.junit.jupiter.api.Test;  
  
public class MultiplyTest {  
  
    Calculator c = new Calculator();  
  
    @Test  
    void testMultiply() {  
        assertEquals(15, c.multiply(3,5));  
    }  
}
```

AllTests.java

```
import org.junit.platform.suite.api.SelectClasses;  
import org.junit.platform.suite.api.Suite;  
@Suite  
@SelectClasses({  
    AddTest.class,  
    MultiplyTest.class  
})  
public class AllTests {  
}
```

Exercise 3: Test Execution Order

Scenario:

You want to control the order in which tests are executed.

Steps:

1. Create a test class `OrderedTests`.
2. Use JUnit's `@TestMethodOrder` and `@Order` annotations.

OrderedTests.java

```
import org.junit.jupiter.api.Order;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestMethodOrder;
import org.junit.jupiter.api.MethodOrderer.OrderAnnotation;
@TestMethodOrder(OrderAnnotation.class)
public class OrderedTests {
    @Test
    @Order(1)
    void firstTest() {
        System.out.println("First Test");
    }
    @Test
    @Order(2)
    void secondTest() {
        System.out.println("Second Test");
    }
    @Test
    @Order(3)
    void thirdTest() {
        System.out.println("Third Test");
    }
}
```

OUTPUT:

First Test

Second Test

Third Test

Exercise 4: Exception Testing

Scenario: You want to test that a method throws the expected exception.

Steps:

1. Create a class `ExceptionThrower` with a method `throwException`.
2. Write a test class `ExceptionThrowerTest` that tests the method for the expected Exception.

ExceptionThrower.java:

```
public class ExceptionThrower {
    public void throwException() {
        throw new ArithmeticException("Division by zero");
    }
}
```

```

    }}
ExceptionThrowerTest.java:
import static org.junit.jupiter.api.Assertions.assertThrows;
import org.junit.jupiter.api.Test;
public class ExceptionThrowerTest {
    ExceptionThrower obj = new ExceptionThrower();
    @Test
    void testException() {
        assertThrows(ArithmeticException.class, () -> {
            obj.throwException();
        });
    }
}

```

Exercise 5: Timeout and Performance Testing

Scenario:

You want to ensure that a method completes within a specified time limit.

Steps:

1. Create a class `PerformanceTester` with a method `performTask`.
2. Write a test class `PerformanceTesterTest` that tests the method for timeout.

PerformanceTester.java:

```

public class PerformanceTester {
    public void performTask() {
        for(int i = 0; i < 100000; i++) {
        }
    }
}

```

PerformanceTesterTest.java:

```

import static org.junit.jupiter.api.Assertions.assertTimeout;
import java.time.Duration;
import org.junit.jupiter.api.Test;
public class PerformanceTesterTest {
    PerformanceTester obj = new PerformanceTester();
    @Test
    void testPerformance() {
        assertTimeout(Duration.ofSeconds(1), () -> {
            obj.performTask();
        });
    }
}

```